

在实际代码部署(如使用 Hugging Face 的 PEFT 库)时, 设置 LoRA 的核心参数秩(Rank r)是平衡显存消耗与模型学习能力的关键。

秩 r 决定了新增的降维矩阵 A 和升维矩阵 B 的内部维度大小。 r 越大, 矩阵能容纳的信息(参数量)就越多, 模型的拟合能力越强, 但同时也会增加梯度和优化器状态的显存开销; r 越小, 显存越省, 但模型可能学不到复杂的特定任务逻辑。

根据课程资料的工业实践, 具体的设置策略可以分为以下几个维度:

1. 根据硬件条件(显存/内存)设置 r 值

- 显存受限 / CPU 微调(如 8GB 左右或纯内存环境): 通常采用较小的秩, 例如设置 $r=8$ 。在这种严苛环境下, 极小的 r 能保证模型顺利跑通, 虽然拟合上限相对较低, 但对于简单的格式对齐或风格模仿任务已经够用。
- 显存充足 / GPU 微调(如 RTX 4090 或专业计算卡): 可以将秩设置得更大, 例如 $r=16$ 甚至更大。更大的秩允许模型捕捉更复杂的垂直领域知识和深层次的推理逻辑。

2. 配合目标模块(Target Modules)的覆盖策略

r 的大小只是其中一个变量, LoRA 作用于模型的哪些层, 对显存和能力的影响同样巨大:

- 低配策略(配合小 r): 在内存受限的情况下, 通常受限仅针对注意力层(Attention 层, 如 `q_proj`, `v_proj`)应用 LoRA。
- 高配策略(配合大 r): 在 GPU 环境下, 不仅 r 可以设得更大, LoRA 还可以全面覆盖多层感知机层(MLP 全连接层, 如 `gate_proj`, `up_proj`)。全面覆盖能显著提升模型对复杂任务的理解力。

3. 总体参数量的“安全水位”

无论你怎么调整 r 值和覆盖层数, 一个核心的经验法则是: 通过 LoRA 引入的可训练参数量, 通常应控制在原模型总参数量的 1%~10% 之间。

- 只要在这个范围内, 优化器状态和梯度带来的显存占用就会被降到极低。以 7B 模型为例, 控制在 1% 左右时, 这部分显存开销仅需约 0.7GB。

总结建议:

在开始一个新的微调项目时, 建议先从 $r=8$ 且仅覆盖 Attention 层(如 `q_proj`, `v_proj`)作为基线(Baseline)跑通流程。如果评估后发现模型“学不会”(欠拟合), 再逐步将 r 提升至 16 或 32, 并把 `gate_proj`、`up_proj` 等 MLP 层加进来, 直到触碰你的显卡显存上限。